

SECURE SOFTWARE ENGINEERING LAB EXAM

20CYS401

NAME:N.ABHINAIREDDY

REGNO:CH.SC.U4CYS23029

Software Requirements Specification and Secure Engineering Report

Zero Trust Enterprise Access Portal

Complete examination submission aligned with all 16 assessed phases

Document control	Value
Document ID	ZTAP SSE 001
Version	2.0
Date	8 October 2026
Scope	Requirements, design, security engineering, Scrum, implementation, testing and deployment evidence
Project	Zero Trust Enterprise Access Portal

Executive summary

This report applies the complete 16 phase Secure Software Engineering laboratory workflow to one consistent system: the Zero Trust Enterprise Access Portal. The portal verifies identity, server side authorization, device posture, policy constraints and explainable risk on every access request. Each phase is derived from the preceding phase and is traceable to implementation and test evidence.

Assessment alignment

Phase	Activity	Marks	Evidence in this report
1	Agile Process	6	Scrum with XP, principles, refactoring and mitigations
2	Requirements Engineering	7	Stakeholders and prioritized SRS
3	Requirements Analysis and UML	7	Use cases and scenario analysis
4	Data and Information Flow	7	ER, DFD and trust boundaries
5	Architecture and Design	7	Architecture, components and patterns
6	User Interface Design	5	Four or more role screens and rationale
7	Threat Modeling and Security	10	Assets, STRIDE, information flow and vulnerabilities
8	Attack Tree and Refinement	6	Attack tree and refined controls
9	Product Backlog and Jira	7	12 stories, Jira hierarchy and two sprint plans
10	Sprint Execution and Metrics	7	Workflow, Scrum, burndown, velocity and review
11	Secure Development	6	Repository, secrets, dependencies and SAST
12	Secure Coding and Refactoring	4	Modules, fixes and secure handling
13	Docker and Kubernetes	7	Container and orchestration security
14	CI CD and Security Testing	7	Pipeline, tests, fuzzing and defects
15	Logging Monitoring and Hardening	5	Events, alerts, hardening and deployment
16	Final Security Review	1	End to end traceability and remaining risks

Contents

- Phase 1 Agile Process
- Phase 2 Requirements Engineering
- Phase 3 Requirements Analysis and UML
- Phase 4 Data and Information Flow
- Phase 5 Architecture and Design
- Phase 6 User Interface Design
- Phase 7 Threat Modeling and Security
- Phase 8 Attack Tree and Refinement
- Phase 9 Product Backlog and Jira
- Phase 10 Sprint Execution and Metrics
- Phase 11 Secure Development
- Phase 12 Secure Coding and Refactoring
- Phase 13 Docker and Kubernetes
- Phase 14 CI CD and Security Testing
- Phase 15 Logging Monitoring and Hardening
- Phase 16 Final Security Review
- Appendices Tools technologies and verification summary

Phase 1 Agile Process and Development Approach

Marks 6 Deliverable Agile approach, Manifesto mapping, refactoring evidence and risk mitigations

1. Agile Methodology Selection

For the ZTAP project, we have selected a hybrid Agile methodology combining Scrum with selected eXtreme Programming (XP) practices.

Selected XP Practices:

- **Pair Programming:** For critical components like the Policy Engine and Authentication Service to ensure immediate peer review of security-critical logic.
- **Test-Driven Development (TDD):** Ensuring that all access control rules, risk assessments, and authentication flows are rigorously tested before implementation.
- **Continuous Integration (CI):** Automated builds and security testing (SAST, dependency scanning) on every commit.
- **Refactoring:** Continuous improvement of code structure without altering external behavior, crucial for maintaining a clean security posture.

Why this is suitable for a security-critical application:

Scrum provides a structured framework for managing complexity and delivering iterative value through Sprints. However, standard Scrum lacks specific engineering practices. By incorporating XP practices like TDD and Pair Programming, we introduce rigorous quality control and security oversight at the code level. This hybrid approach ensures that we can respond to changing business needs (Scrum) while maintaining the high assurance and low defect rate required for an enterprise zero-trust portal (XP).

2. Agile Manifesto Principles Mapped to ZTAP

1. Individuals and interactions over processes and tools:
 - ***ZTAP Mapping:*** Fostering close collaboration between the development team and the security team on threat models and architectural reviews, ensuring security is built-in rather than bolted-on by a tool later.
2. Working software over comprehensive documentation:
 - ***ZTAP Mapping:*** Prioritizing demonstrable features, such as a working Policy Engine making ALLOW/DENY decisions, over lengthy theoretical specifications. (Though security docs remain essential, the focus is on the working system).
3. Customer collaboration over contract negotiation:
 - ***ZTAP Mapping:*** Active involvement of stakeholders (Security Administrators, Resource Owners) in defining and refining security requirements, risk thresholds, and access workflows.
4. Responding to change over following a plan:
 - ***ZTAP Mapping:*** The ability to rapidly adapt the Risk Engine and Policy Engine to the evolving threat landscape and new attack vectors identified during the project lifecycle.
5. Simplicity - the art of maximizing the amount of work not done - is essential:
 - ***ZTAP Mapping:*** Adhering to the principle of "least-privilege by default." Keeping access rules, policy definitions, and the codebase as simple as possible to reduce the attack surface and minimize potential vulnerabilities.

3. Refactoring Opportunities

Refactoring 1: Inline Authorization Checks → Middleware-based RBAC

- **Before:** Each API route manually checks user roles (e.g., if (user.role === 'admin')). This approach is error-prone; developers might forget the check on a new endpoint.

- After: Implementation of a withAuthorization('SECURITY_ADMIN') middleware wrapper that centrally intercepts requests.
- Security improvement: Eliminates the risk of missed authorization checks on critical endpoints (broken access control).
- Maintainability improvement: Provides a single point of change for role-evaluation logic.

Refactoring 2: Direct Password Handling → Dedicated AuthService

- Before: Password hashing and comparison logic are implemented inline within the login and registration API routes.
- After: Encapsulating all password operations into a dedicated AuthService.verifyCredentials() method using Argon2id.
- Security improvement: Ensures consistent hashing parameters across the application and prevents accidental logging or exposure of plaintext passwords in route handlers.
- Maintainability improvement: Creates a single, highly cohesive module for all credential handling, simplifying future updates (e.g., changing work factors).

4. Agile Limitations and Risks for Security-Critical Systems

- Risk 1: Fast iterations and pressure to deliver features quickly may lead to skipping thorough security reviews or penetration testing.
- Mitigation: Enforce a mandatory security review checklist per sprint. Integrate automated security testing (SAST/DAST) into the CI pipeline to catch low-hanging fruit automatically.
- Risk 2: Changing requirements and constant refactoring may invalidate the initial threat model, introducing unforeseen attack vectors.
- Mitigation: Make a lightweight threat model review a mandatory component of the "Definition of Done" for any architectural or significant feature change.

Phase 2 Requirements Engineering

Marks 7 Deliverable Stakeholders and concise functional, nonfunctional and security requirements

1. Stakeholders

Stakeholder	Description	Key Concerns
Employee/User	Standard user requesting access to enterprise resources.	Ease of use, quick access approvals, clear feedback on access denial.
Security Administrator	Manages system-wide security policies, roles, and device trust configurations.	Comprehensive visibility, fine-grained policy enforcement, incident response capabilities.
Resource Owner	Responsible for specific enterprise resources and defining access criteria.	Granular control over resource access, delegation of approval authority.
Auditor	Independent entity verifying compliance with security standards and regulations.	Immutable audit logs, transparent access decisions, historical reporting.
System Administrator	Manages the underlying infrastructure (Docker, K8s, PostgreSQL).	System uptime, scalability, easy deployment, backup and recovery.

2. Functional Requirements

ID	Requirement Description	Priority
FR-01	The system must allow Users to authenticate using a secure mechanism.	Must
FR-02	The system must provide Security Administrators with a dashboard to manage Users and Roles.	Must
FR-03	The system must allow Resource Owners to register and manage Resources.	Must
FR-04	The system must track Device Trust state (TRUSTED, COMPLIANT, UNTRUSTED, COMPROMISED).	Must
FR-05	The system must allow Users to submit Access Requests for specific Resources.	Must
FR-06	The Policy Engine must evaluate access requests based on Identity, Role, Device Trust, Resource Policy, and Time.	Must
FR-07	The Risk Engine must calculate a risk score (0-100) for each access request.	Should
FR-08	The system must support Access Decisions of ALLOW, DENY, STEP_UP_AUTHENTICATION, and REQUIRE_APPROVAL.	Must

ID	Requirement Description	Priority
FR-09	The system must route REQUIRE_APPROVAL decisions to the designated Resource Owner.	Should
FR-10	The system must generate immutable Audit Logs for all authentication attempts and access decisions.	Must
FR-11	The system must provide role-specific Dashboards tailored to the logged-in user.	Should
FR-12	The system must manage active Sessions and allow Security Admins to terminate them.	Must
FR-13	The system must implement account lockout mechanisms after consecutive failed login attempts.	Must
FR-14	The system must allow Security Administrators to assign multi-role mappings to Users.	Should
FR-15	The system must provide search and filter capabilities for Audit Logs and Access Requests.	Could

3. Non-Functional Requirements

ID	Requirement Description	Priority
NFR-01	Performance: The Policy Engine must evaluate access requests in under 200ms.	Must
NFR-02	Scalability: The system must support horizontal scaling to handle 10,000 concurrent users.	Should
NFR-03	Availability: The system must achieve 99.9% uptime during business hours.	Must
NFR-04	Usability: The user interface must comply with WCAG 2.1 AA accessibility standards.	Should
NFR-05	Maintainability: The codebase must maintain a minimum test coverage of 85%.	Must
NFR-06	Portability: The application must be deployable via Docker and Kubernetes.	Must
NFR-07	Compatibility: The web frontend must support the latest versions of Chrome, Firefox, Safari, and Edge.	Should
NFR-08	Documentation: All API endpoints must be documented using OpenAPI/Swagger specifications.	Should

4. Security Requirements

ID	Requirement Description	Priority
SR-01	Confidentiality: All sensitive data in transit must be encrypted using TLS 1.2 or higher.	Must
SR-02	Authentication: Passwords must be hashed using the Argon2id algorithm; plaintext storage is strictly forbidden.	Must
SR-03	Authorization: All API endpoints must implement server-side Role-Based Access Control (RBAC).	Must
SR-04	BOLA Protection: The system must verify that the requesting user has authorization to access the specific requested entity ID (Prevent IDOR/BOLA).	Must
SR-05	Input Validation: All user inputs must be strictly validated and sanitized on the backend prior to processing.	Must
SR-06	Session Security: Session tokens must be securely generated, stored in HttpOnly/Secure cookies, and have a defined expiration time.	Must
SR-07	CSRF Protection: The system must implement anti-CSRF tokens for all state-changing operations.	Must
SR-08	Security Headers: HTTP responses must include appropriate security headers (e.g., CSP, X-Frame-Options, HSTS).	Must
SR-09	Audit Logging: Audit logs must be tamper-evident and append-only at the application level.	Must
SR-10	Secrets Management: Cryptographic keys and application secrets must be stored securely (e.g., using a vault or environment variables), never in code.	Must
SR-11	Privilege Escalation: System architecture must prevent vertical and horizontal privilege escalation attacks.	Must
SR-12	Device Trust: The system must verify Device Trust status before evaluating access policies.	Must
SR-13	Least Privilege: Internal services and database users must operate with the minimum privileges necessary.	Must

Phase 3 Requirements Analysis and UML

Marks 7 Deliverable Use case diagram, two specifications and scenario analysis

Use Case Model

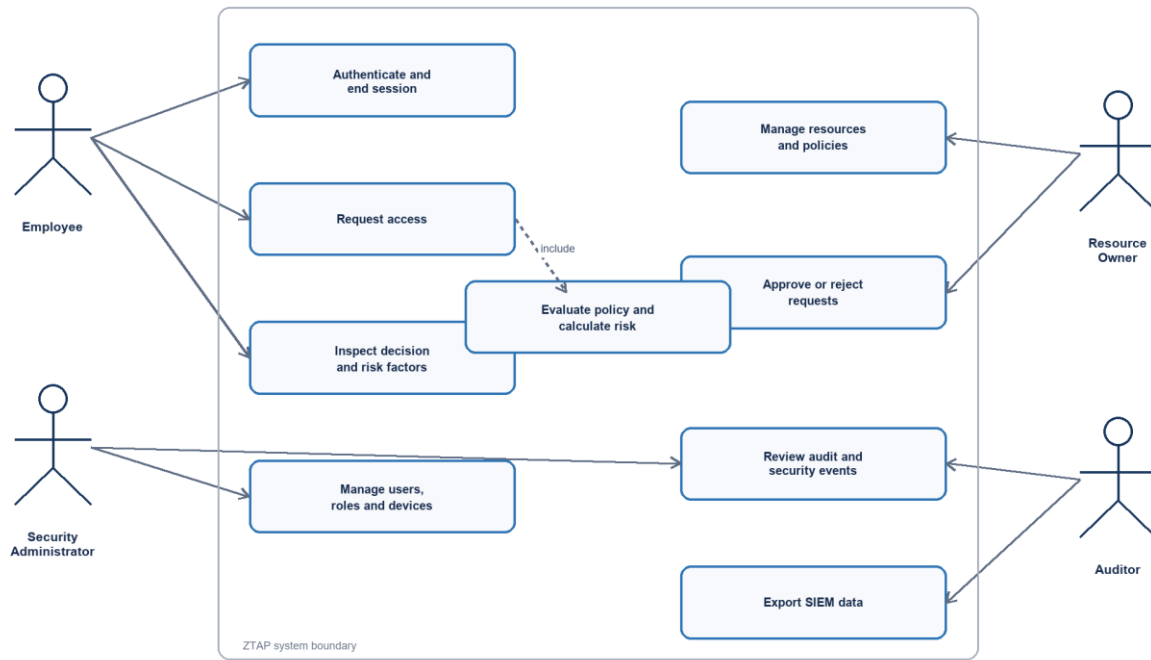


Figure 1 UML use case diagram

1. Use Case Diagram

The Zero-Trust Access Portal (ZTAP) use cases encompass actions taken by four core actors: Employee (User), Security Administrator, Resource Owner, and Auditor.

2. Critical Use Case 1: Request Access to Internal Application

- Actor: Employee
- Preconditions: User is authenticated, device is registered, resource exists and is active.
- Main Flow:
 6. Employee selects a targeted resource.
 7. Employee enters the justification or reason for access.
 8. Employee submits the access request.
 9. System captures current device context and information.
 10. Policy engine evaluates the request against RBAC and ABAC rules.
 11. Risk engine generates a risk score (0-100).
 12. Access decision is returned to the user.
 13. Audit log entry is created.
- Alternative Flow:
 - *Step-Up Auth*: If Risk Score is 60-80, the system sets status to STEP_UP_AUTHENTICATION requiring MFA.
 - *Approval Needed*: If the Resource is highly sensitive and requires manual override, the status is set to REQUIRE_APPROVAL.
- Exception Flow:
 - *Compromised Device*: If device trust evaluates to COMPROMISED, immediate DENY.

- ***Inactive User*:** If user account is suspended or inactive, immediate DENY.
- ***Missing Resource*:** If the requested resource is inactive or removed, system throws an error.
- **Postconditions:** An AccessRequest entity is created, an AccessDecision is recorded, and an AuditLog entry is firmly stored.

3. Critical Use Case 2: Evaluate Access Request

- Actor: System (Policy Engine)
- Preconditions: A valid AccessRequest payload is received.
- Main Flow:
 14. Validate the user identity and session token.
 15. Check role authorization via RBAC policies.
 16. Evaluate device trust (TRUSTED, COMPLIANT, UNTRUSTED, COMPROMISED).
 17. Check resource sensitivity and access constraints.
 18. Evaluate temporal conditions (e.g., business hours restrictions).
 19. Calculate the aggregate risk score via the Risk Engine.
 20. Aggregate all evaluations to formulate a final ALLOW, DENY, STEP_UP_AUTHENTICATION, or REQUIRE_APPROVAL decision.
 21. Generate a detailed decision reason string.
- Alternative Flow:
 - ***Missing Device Info*:** System assigns maximum device risk (100) or fails open/closed depending on global strictness.
 - ***Implicit Role Allowance*:** If a role is not explicitly allowed, check for wildcard access policies or organizational defaults.
- Exception Flow:
 - ***Policy Engine Error*:** Output DENY securely and log an internal error alert.
 - ***Database Unavailable*:** Output DENY securely and surface a system error gracefully to the user.
- **Postconditions:** AccessDecision generated comprising the decision flag, string reason, calculated risk score, and detailed contributing factors.

4. Scenario-Based Analysis: "Employee Requests Access to Sensitive Internal Application"

- Scenario: John (Employee) requests access to the Finance System (HIGH sensitivity) from his managed laptop during business hours.
- Walkthrough:
 22. Initiation: John logs into ZTAP, navigates to his dashboard, and requests access to the "Finance System".
 23. Data Capture: ZTAP captures John's identity, his laptop's device signature, and the current timestamp.
 24. Device Trust Check: The laptop is verified against the Device Store and marked as TRUSTED and COMPLIANT.
 25. Role Evaluation: John's role (Finance Analyst) is evaluated. The policy confirms the role has baseline permission for the system.
 26. Risk Calculation: Because the device is TRUSTED, the time is within normal business hours, and the location is recognized, the Risk Engine scores the transaction at 15 (Low Risk).
 27. Policy Decision: The Policy Engine aggregates these vectors. Since the risk is below 60 and RBAC aligns, the system grants an ALLOW decision.
 28. Audit & Fulfillment: An AccessDecision is saved to the database and John is securely routed to the Finance System interface.

Phase 4 Data and Information Flow Modeling

Marks 7 Deliverable ER diagram and Level 0 and Level 1 DFD with trust boundaries

Entity Relationship Diagram

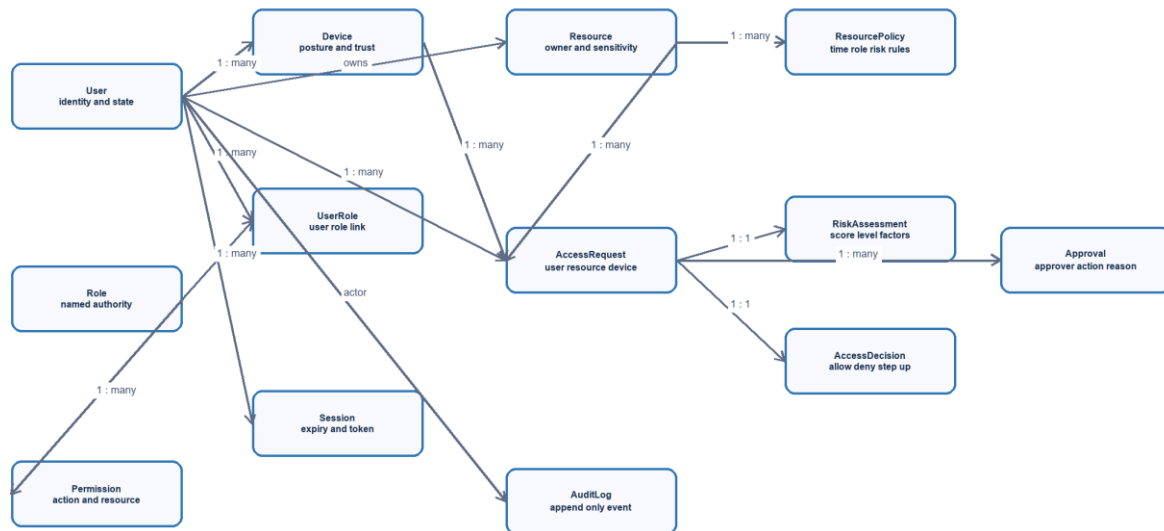
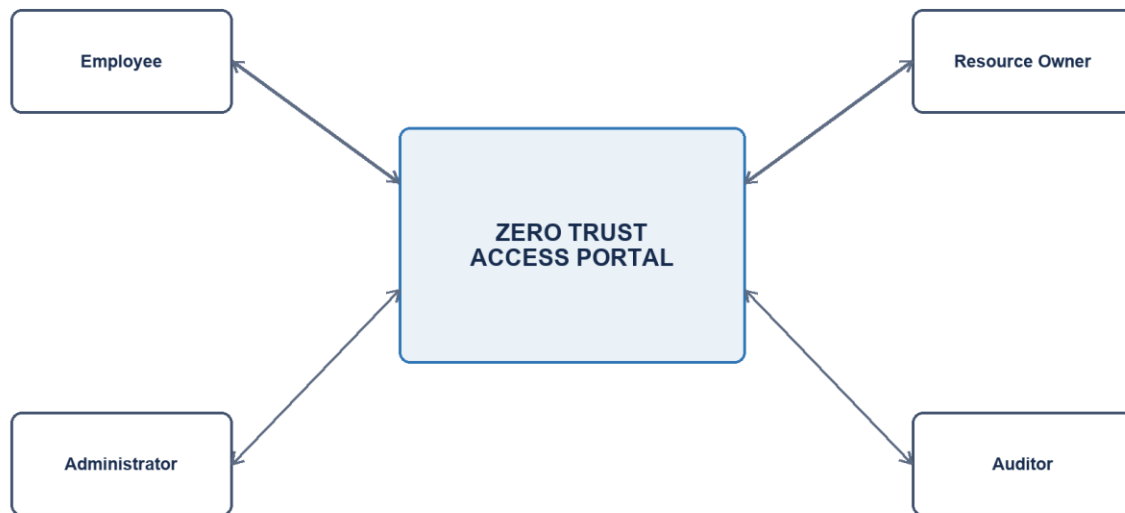


Figure 2 Entity relationship diagram

Zero Trust Access Portal DFD Level 0



Trust boundary: all external requests are untrusted until authenticated, authorized and evaluated

Figure 3 Data flow diagram Level 0

Data Flow Diagram Level 1

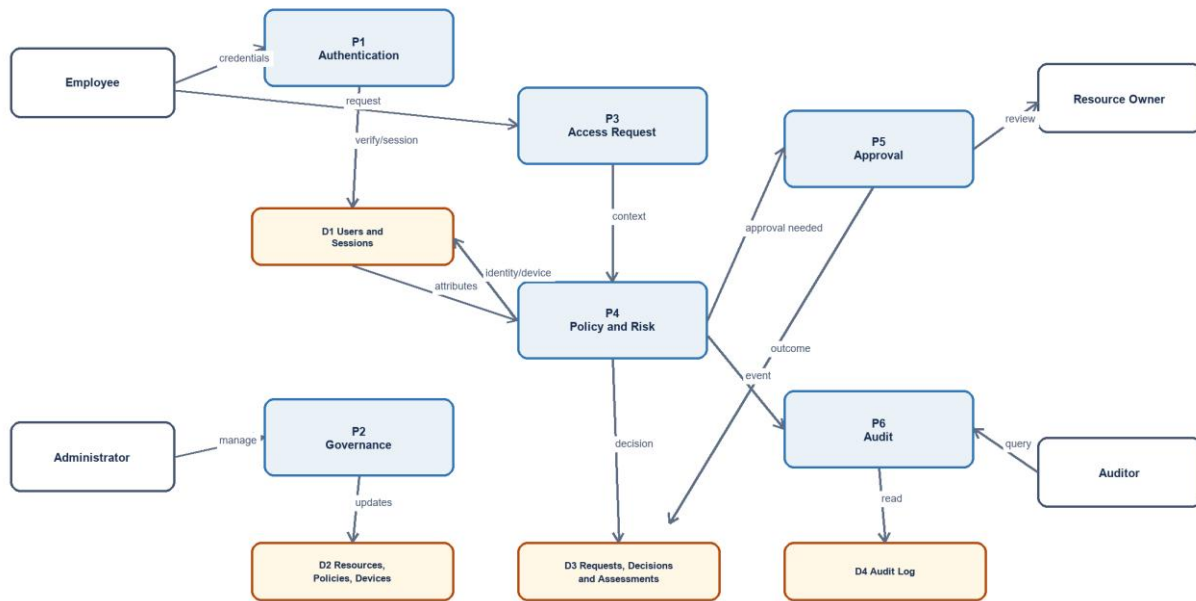


Figure 4 Data flow diagram Level 1

1. Entity-Relationship (ER) Diagram

2. DFD Level 0 (Context Diagram)

The Level 0 Data Flow Diagram outlines the highest level of abstraction for the Zero-Trust Access Portal (ZTAP).

3. DFD Level 1

The Level 1 DFD decomposes the ZTAP system into primary functional processes and associated data stores.

4. Trust Boundaries

Trust boundaries delineate different levels of trust within the application's architecture:

- TB1: User Browser → Frontend: Low-trust boundary. Input must be validated. XSS/CSRF protections are necessary.
- TB2: Frontend → Backend API: Crossing from the public network into the internal network structure. Requires strict authentication, rate-limiting, and CORS controls.
- TB3: Backend → Database: High-trust inner boundary. Protects against SQLi using an ORM (Prisma). Minimal privileges assigned to the DB credentials.
- TB4: Backend → Policy Engine: Internal integration point evaluating zero-trust rules. Output determines access state, so responses must be cryptographically verified if decoupled.
- TB5: Admin Interface → Security Management APIs: Privileged boundary. Strict RBAC enforcement and heightened audit logging apply to all administrative interactions.

5. Consistency Check

- Use Cases mapped to DFD Processes:
- Login is executed in P1 Authentication.
- Request Access triggers P4 Access Request Processing.
- Evaluate Access and Calculate Risk are handled by P5 Policy Evaluation and P6 Risk Assessment.
- Manage Users, Roles, Policies, and Resources correspond directly to P2 User Management, P3 Resource Management, and associated stores.
- Use Cases mapped to ER Entities:

- Initiating an Access Request creates an AccessRequest entity.
- Policy execution pulls from ResourcePolicy, Role, UserRole, Device, and creates an AccessDecision and RiskAssessment.
- Entity consistency: All items inside D1 User Store, D2 Resource Store, etc., map exactly to the entities detailed in the ER diagram ensuring the data and workflow maintain a 1-to-1 parity structurally.

Phase 5 Software Architecture and Design Engineering

Marks 7 Deliverable Architecture diagram, component design and justification

Zero Trust Access Portal Architecture

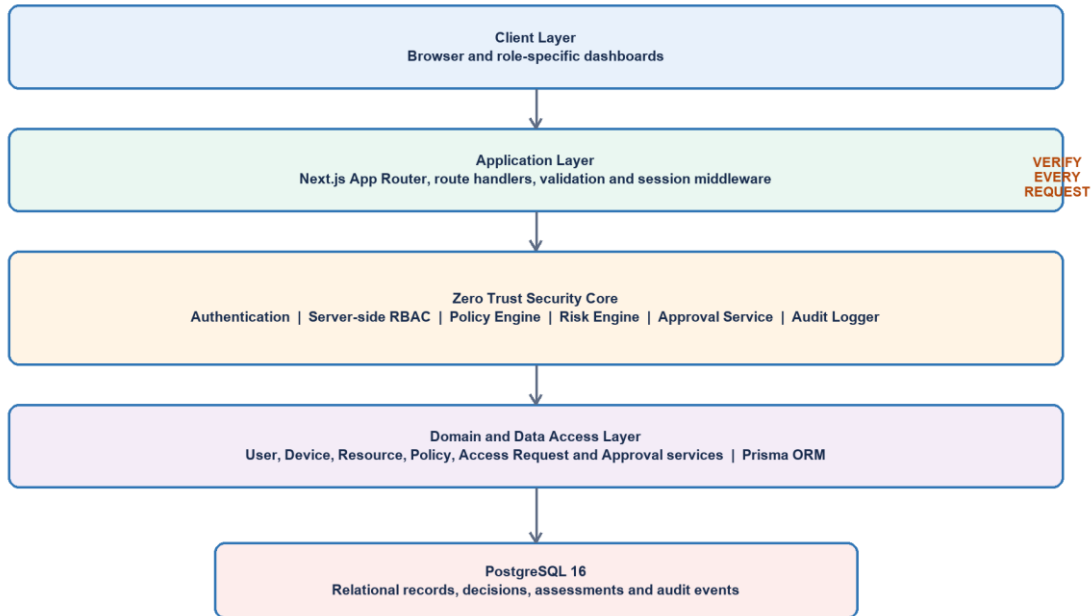


Figure 5 Zero Trust layered project architecture

Secure Layered Architecture

The Zero-Trust Enterprise Access Portal (ZTAP) utilizes a secure layered architecture to separate concerns, maintain trust boundaries, and enforce zero-trust principles at every level.

- **Presentation Layer:** Next.js pages, React components (Client Components and Server Components), and shadcn/ui elements. Responsible for rendering the UI and handling user interactions.
- **API Layer:** Next.js Route Handlers exposing RESTful endpoints. Responsible for request parsing, validation (Zod), and initial routing.
- **Authentication Layer:** Manages sessions using iron-session (encrypted, HTTP-only cookies) and handles password hashing/verification using Argon2id. Enforces MFA checks.
- **Authorization Layer:** RBAC middleware and fine-grained permission checks. Ensures the authenticated user possesses the correct roles and permissions before allowing business logic execution.
- **Policy Engine:** A strategy-based evaluation pipeline that applies ResourcePolicy rules against incoming AccessRequests. Evaluates identity, device trust, resource sensitivity, time, and environment context.
- **Risk Engine:** Factor-based risk calculation module. Evaluates anomalous behaviors, location, device health, and historical access patterns to generate a risk score (0-100).
- **Business Logic Layer:** Core services (UserService, AccessRequestService) encapsulating the domain rules and workflows.
- **Audit Service:** Immutable logging service recording all security-relevant events, access decisions, and state changes for compliance and analysis.
- **Data Access Layer:** Prisma ORM interacting with PostgreSQL. Responsible for data persistence, transaction management, and query optimization.

Design Patterns

ZTAP employs several well-established design patterns to ensure maintainability and security.

- **Repository Pattern:** UserRepository and ResourceRepository abstract database operations via Prisma, allowing the business logic to remain agnostic of the underlying ORM and facilitating unit testing.
- **Service Layer Pattern:** Core logic is encapsulated in services like AccessRequestService, coordinating interactions between the repository, Policy Engine, and Audit Service.
- **Strategy Pattern:** The Policy Engine employs interchangeable evaluators (IdentityEvaluator, DeviceEvaluator, TimeEvaluator). This allows the system to evaluate diverse policies dynamically without modifying core engine logic.
- **Factory Pattern:** A RiskCalculatorFactory dynamically instantiates the appropriate risk calculators based on the user's context and the resource type being accessed.
- **Middleware Pattern:** Cross-cutting security concerns (Authentication validation, RBAC checks, Rate Limiting, Security Headers) are implemented as composable Next.js middleware, ensuring they are executed prior to reaching API endpoints.
- **Policy Object / Specification Pattern:** The ResourcePolicy entity encapsulates complex access rules as composable conditions (e.g., RequiresTrustedDevice AND IsDuringBusinessHours), separating policy definition from policy evaluation.

Architecture Diagram

Component Diagram

Sequence Diagram: Access Request Flow

Technology Justification

Technology	Purpose	Justification
Next.js	Full-stack React Framework	Provides SSR/SSG for performance, API routes for backend logic, and robust file-based routing. Simplifies deployment.
TypeScript	Language	Enforces strict type safety across the stack, reducing runtime errors and improving developer experience and code maintainability.
PostgreSQL	Relational Database	Offers ACID compliance, structured schema, and robust indexing essential for transactional consistency in access management and auditing.
Prisma ORM	Data Access	Provides a type-safe database client. Abstracts complex SQL queries and integrates seamlessly with TypeScript.
Tailwind CSS	Styling	Utility-first CSS framework enabling rapid, consistent UI development without context switching.
shadcn/ui	UI Components	Accessible, customizable component library that provides a professional look and feel without vendor lock-in.
Argon2id	Password Hashing	Current industry standard for secure password hashing. Resistant to GPU cracking and side-channel attacks.
iron-session	Session Management	Stateless, encrypted, HTTP-only cookie-based sessions. Prevents database lookups for session

Technology	Purpose	Justification
		validation and mitigates CSRF.
Docker / K8s	Containerization & Orchestration	Ensures consistent environments from development to production. Kubernetes provides scaling and high availability.

Phase 6 User Interface Design

Marks 5 Deliverable Role based wireframes and design rationale

Role Based Interface Wireframes

The figure displays four wireframe panels for different user roles in the Zero Trust Enterprise Access Portal:

- Login:** Contains fields for Email, Password, a Sign in button, and a Clear error and logout feedback button.
- Employee Dashboard:** Contains sections for My devices, Protected applications, Request access, and Decision and risk explanation.
- Resource Owner:** Contains sections for Pending approvals, Requester and resource, Risk factors, and an Approve / Reject / Step up button.
- Administrator and Auditor:** Contains sections for Users and roles, Device posture, Policies, and a Searchable audit trail.

Figure 6 Login, employee, owner, administrator and auditor wireframes

Core Usability Principles Applied

- **Consistency:** Uniform layout utilizing a standard sidebar navigation, consistent color palette, and reusable components (shadcn/ui).
- **User control:** Prominent "Cancel" and "Back" buttons on forms. Destructive actions require explicit confirmation.
- **Feedback:** Global loading indicators, skeleton screens for asynchronous data, and toast notifications for success/error events.
- **Error prevention:** Client-side form validation via Zod, clear input requirements, and confirmation dialogs for critical actions.
- **Navigation:** Role-based sidebar menu, breadcrumbs indicating current location, and clear page titles.
- **Visibility:** Access decisions clearly display risk scores, security levels, and the reasoning behind decisions.

Navigation Flow

Screen Documentation

1. Login Page

- **User/Role:** All users.
- **Goal:** Authenticate users securely into the ZTAP portal.
- **Navigation:** Landing page (/login). Upon success, redirects to the role-appropriate dashboard.
- **Inputs:** Username/Email, Password, optionally MFA token.
- **Feedback:** Loading spinner on submit.
- **Errors:** "Invalid credentials", "Account locked", validation errors for empty fields.

- Security considerations: Generic error messages to prevent username enumeration. Rate-limited to prevent brute force attacks.

2. Employee Dashboard

- User/Role: Employee/User.
- Goal: Overview of active sessions, recent access requests, and quick links to request new access.
- Navigation: Accessed via /dashboard. Links to Request Access Form and specific Access Decision Details.
- Inputs: Filter/Search recent requests.
- Feedback: Skeleton loaders for initial data fetch.
- Errors: Network errors display retry button.
- Security considerations: Only shows data owned by the logged-in user. Strict tenant isolation.

3. Request Access Form

- User/Role: Employee/User.
- Goal: Submit a structured request for access to a specific enterprise resource.
- Navigation: Accessed from Employee Dashboard. Submits to Access Decision Details upon completion.
- Inputs: Resource dropdown/search, Justification text area, Duration requested.
- Feedback: Toast notification on successful submission. Loading state on button.
- Errors: Client-side validation for required fields.
- Security considerations: Resource list is pre-filtered to only show resources the user is permitted to request.

4. Access Decision Details

- User/Role: Employee, Security Administrator, Resource Owner.
- Goal: Display the outcome of an access request, including policy reasoning and risk score.
- Navigation: Reached from Dashboard or Approval Queue.
- Inputs: None for Employee. Owners/Admins may have "Approve/Deny" action buttons.
- Feedback: Visual indicators for ALLOW (green), DENY (red), PENDING (yellow).
- Errors: "Access Denied" if attempting to view an unauthorized request ID.
- Security considerations: IDOR prevention ensures users can only view their own requests, unless they hold an Admin/Owner role managing that specific request.

5. Security Administrator Dashboard

- User/Role: Security Administrator.
- Goal: High-level overview of system security posture, active alerts, and quick management links.
- Navigation: Accessed via /admin/dashboard. Links to management pages.
- Inputs: Date range filters for charts.
- Feedback: Interactive charts and data refreshing indicators.
- Errors: Standard data fetch errors.
- Security considerations: Route strictly protected by RBAC middleware requiring ADMIN role.

6. User Management

- User/Role: Security Administrator.
- Goal: Manage user accounts, assign roles, and review individual user risk profiles.
- Navigation: Accessed from Admin Dashboard.
- Inputs: Search bar, role filters, user edit forms.
- Feedback: Toast on role update. Confirmation modal before disabling a user.
- Errors: Validation errors on form submission.
- Security considerations: Prevents self-locking or privilege escalation vulnerabilities.

7. Resource Management

- User/Role: Security Administrator, Resource Owner.
- Goal: Register new applications/resources and define their sensitivity levels.
- Navigation: Accessed from Admin Dashboard.
- Inputs: Form for resource name, type, sensitivity classification, owner assignment.

- Feedback: Success toast on resource creation.
- Errors: Name collision errors.
- Security considerations: Resource Owners can only manage resources assigned to them.

8. Policy Management

- User/Role: Security Administrator.
- Goal: Define and manage ResourcePolicies (the rules dictating access).
- Navigation: Accessed from Admin Dashboard.
- Inputs: Complex policy builder UI (conditions, target resources, required trust levels).
- Feedback: Validation feedback on policy syntax/logic.
- Errors: Syntax errors in policy definition.
- Security considerations: Audit logging tracks all policy modifications, detailing who changed what and when.

9. Device Management

- User/Role: Security Administrator.
- Goal: View device inventory, manually override device trust states (TRUSTED, UNTRUSTED, COMPROMISED).
- Navigation: Accessed from Admin Dashboard.
- Inputs: Search by MAC/IP/User. Action buttons to update status.
- Feedback: State change confirmations.
- Errors: Action failure alerts.
- Security considerations: Marking a device as COMPROMISED immediately terminates all active sessions associated with it.

10. Audit Log Viewer

- User/Role: Auditor, Security Administrator.
- Goal: Search, filter, and export immutable security event logs.
- Navigation: Accessed via /audit.
- Inputs: Advanced filtering (date, event type, actor, resource), export buttons (CSV/PDF).
- Feedback: Progress bar during export generation.
- Errors: Query timeout handling.
- Security considerations: Read-only interface. No modification capabilities exist.

11. Resource Owner Approval Queue

- User/Role: Resource Owner.
- Goal: Review and disposition pending access requests for owned resources.
- Navigation: Accessed via /approvals.
- Inputs: Bulk selection checkboxes, Approve/Deny buttons, justification text input for decision.
- Feedback: Toast on successful disposition. Item removed from queue.
- Errors: Concurrency errors if request was handled by another owner.
- Security considerations: Enforces that owners only see requests explicitly requiring their approval.

12. Risk Analysis View

- User/Role: Security Administrator.
- Goal: Deep dive into the Risk Engine's outputs, viewing factor breakdowns for specific users or requests.
- Navigation: Accessed from Admin Dashboard or Access Decision Details.
- Inputs: Filter by risk threshold (e.g., Score > 80).
- Feedback: Detailed breakdown charts of risk factors.
- Errors: Data unavailability notices.
- Security considerations: Exposes sensitive behavioral data; strictly limited to administrators.

Phase 7 Threat Modeling and Security Analysis

Marks 10 Deliverable Asset CIA, STRIDE, information flow and vulnerability analysis

Threat model and STRIDE

1. Assets and CIA Classification

Asset	Confidentiality	Integrity	Availability	Description
User credentials (passwords, hashes)	High	High	High	Hashed passwords (Argon2id) used for user authentication.
Session tokens	High	High	High	Tokens identifying authenticated user sessions.
User identity and profile data	Medium	High	High	Basic user information (email, name).
Role and permission assignments	High	High	High	RBAC mapping for users.
Access policies and rules	High	High	High	Policies determining resource access.
Internal resource/application metadata	Medium	High	High	Details about the protected resources.
Access request data	Medium	High	High	Records of user requests for access.
Audit logs	High	High	High	System-wide logs for accountability.
Risk assessment results	Medium	High	High	Risk Engine evaluation scores (0-100).
Approval decisions	Medium	High	High	Outcomes of access requests (ALLOW/DENY).
Device trust information	Medium	High	High	Device status (TRUSTED/COMPLIANT/UNTRUSTED/COMPROMISED).
Database connection credentials	High	High	High	Credentials for the backend to access PostgreSQL.

2. STRIDE Threat Analysis

ID	DFD Element	Threat	STRIDE Category	Impact	Likelihood	Risk Level	Mitigation
T-01	Login API	Attacker impersonates user with stolen credentials	Spoofing	High	Medium	High	Argon2id hashing, account lockout, rate limiting
T-02	Session Store	Attacker modifies session token	Tampering	High	Low	Medium	Signed encrypted sessions (iron-

ID	DFD Element	Threat	STRIDE Category	Impact	Likelihood	Risk Level	Mitigation
							session), HttpOnly Secure cookies
T-03	Access Request API	User denies making access request	Repudiation	Medium	Medium	Medium	Immutable audit logging with timestamps and correlation IDs
T-04	Policy Engine	Attacker extracts policy rules	Information Disclosure	Medium	Low	Low	Server-side only, no policy details in API responses
T-05	Login API	Brute force attack	Denial of Service	High	High	High	Rate limiting, account lockout, CAPTCHA-ready
T-06	User Management API	Employee escalates to admin role	Elevation of Privilege	Critical	Medium	Critical	Server-side RBAC, role change audit logging
T-07	Access Request API	Bypass device trust check	Elevation of Privilege	High	Medium	High	Server-side device validation, no client-side trust
T-08	Audit Log Store	Attacker deletes audit entries	Tampering	Critical	Low	High	Append-only logging, no DELETE endpoint, DB-level protection
T-09	Resource API	IDOR to access other users' resources	Information Disclosure	High	Medium	High	Server-side ownership checks, parameterized queries
T-10	Approval API	Employee approves own request	Elevation of Privilege	High	Medium	High	Self-approval prevention check, audit logging
T-11	Device API	Spoofed device status report	Spoofing	High	Medium	High	Mutual TLS, cryptographic device attestation
T-12	Database	Unauthorized database extraction	Information Disclosure	Critical	Low	High	Network isolation, strict DB user privileges, encrypted backups

Information flow analysis

Sensitive Asset Analysis

1. Authentication Credentials

- Source: User Browser (Frontend)
- Destination: Authentication Service (Backend)
- Trust Boundary: TB1 (Browser to Network), TB2 (Network to Backend API)
- Transformation: Plaintext (at input) → Argon2id hash (at storage/verification)
- Storage: PostgreSQL users table (passwordHash column)
- Security Controls: HTTPS in transit, Argon2id hashing, no logging of plaintext passwords.

2. Access Policies

- Source: Security Administrator
- Destination: Policy Engine
- Trust Boundary: TB4 (Admin Browser to Network), TB5 (Network to Backend API)
- Transformation: JSON policy rules (UI) → evaluated conditions (Memory)
- Storage: PostgreSQL resource_policies table
- Security Controls: RBAC enforcement, admin-only access, comprehensive audit logging of all policy changes.

3. Audit Logs

- Source: All system components (Auth, Policy Engine, Resource API)
- Destination: Audit Log Store
- Trust Boundary: TB3 (Backend API to Database)
- Transformation: Structured Event Object (Memory) → Immutable Record (Database)
- Storage: PostgreSQL audit_logs table
- Security Controls: Append-only design, removal of sensitive data before logging, tamper-detection mechanisms.

Information Flow Diagram

Vulnerability analysis

Identified Vulnerabilities

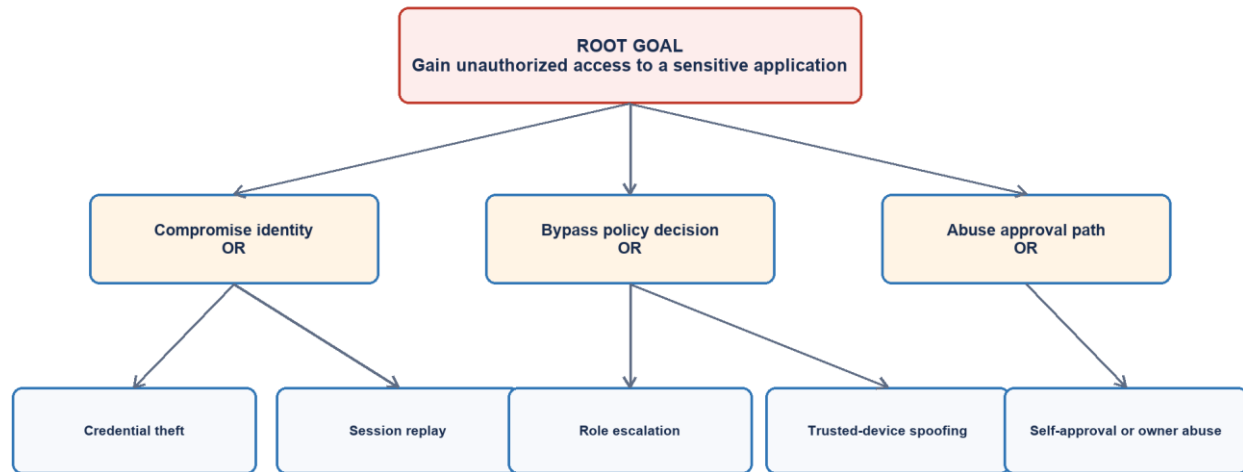
ID	Vulnerability	Affected Component	Threat	Impact	Likelihood	Risk	Mitigation	Verification Test
VULN-01	IDOR/BOLA on access requests	Access Request API	T-09	High	Medium	High	Server-side ownership validation	Test: Employee A cannot view Employee B's requests
VULN-02	Broken access control on user management	User Management API	T-06	Critical	Medium	Critical	RBAC middleware, server-side role check	Test: Employee cannot call POST /api/users
VULN-03	Privilege escalation via role self-assignment	User/Role API	T-06	Critical	Medium	Critical	Prevent users from modifying own roles, audit log	Test: Employee cannot change own role to admin
VULN-04	Session hijacking	Session management	T-02	High	Low	Medium	HttpOnly, Secure, SameSite cookies, session rotation	Test: Cookie cannot be accessed via JS

ID	Vulnerability	Affected Component	Threat	Impact	Likelihood	Risk	Mitigation	Verification Test
VULN-05	SQL injection	Database queries	T-02	Critical	Low	Medium	Prisma ORM (parameterized), no raw SQL	Test: SQL injection payloads return errors
VULN-06	XSS in reason/description fields	Access request form, resource description	T-04	Medium	Medium	Medium	Input sanitization, React auto-escaping, CSP header	Test: Script tags in reason field are escaped
VULN-07	Insecure password storage	Authentication	T-01	Critical	Low	High	Argon2id with proper parameters	Test: Verify passwordHash is Argon2id format
VULN-08	Hard-coded secrets	Configuration	T-04	High	Medium	High	Environment variables, .env.example, .gitignore	Test: grep source for passwords/secrets

Phase 8 Attack Tree and Security Architecture Refinement

Marks 6 Deliverable Attack tree, attack paths, controls and refined architecture

Attack Tree for Unauthorized Access



Controls: Argon2id + lockout | encrypted secure sessions | server-side RBAC | device posture | segregation of duties | immutable audit alerts

Figure 7 Attack tree with preventive and detective controls

Primary Goal: Gain Unauthorized Access to Sensitive Application

```

GOAL: Gain Unauthorized Access to Sensitive Application
├── OR: Steal Valid Credentials
│   ├── Brute force login
│   ├── Phishing attack
│   └── Credential stuffing
├── OR: Exploit Session
│   ├── Session hijacking (XSS)
│   ├── Session fixation
│   └── Replay attack
├── OR: Privilege Escalation
│   ├── AND: Modify own role
│   │   ├── Find unprotected role API
│   │   └── Assign admin role to self
│   └── AND: Exploit IDOR
│       ├── Discover other user's request ID
│       └── Modify request to approve own access
├── OR: Bypass Policy Engine
│   ├── Manipulate device trust data
│   ├── Manipulate timestamp
│   └── Access resource directly bypassing API
└── OR: Compromise Trusted Device
    ├── AND: Install malware
    │   ├── Gain access to employee device
    │   └── Modify device trust status
  
```

Path Defenses (Primary Goal)

29. Steal Valid Credentials

- Preventive Controls: Argon2id hashing, rate limiting, MFA/Step-up authentication.
- Detective Controls: Failed login monitoring, abnormal geographic login alerts.
- Mitigations: Account lockout, forced password reset on suspected compromise.

30. Exploit Session

- Preventive Controls: HttpOnly, Secure, SameSite=strict cookies, short session lifespans.
- Detective Controls: Concurrent session detection.
- Mitigations: Immediate session invalidation.

31. Privilege Escalation

- Preventive Controls: Strict server-side RBAC, self-modification restrictions.
- Detective Controls: Audit log alerts for role changes.
- Mitigations: Revert unauthorized changes, suspend offending accounts.

32. Bypass Policy Engine

- Preventive Controls: Default-deny posture, API gateway enforcing policy evaluation, backend resource isolation.
- Detective Controls: Access requests lacking corresponding policy evaluation records.
- Mitigations: Block direct resource access network-wide.

33. Compromise Trusted Device

- Preventive Controls: EDR (Endpoint Detection and Response) agent, OS updates.
- Detective Controls: Device trust status downgrade signals.
- Mitigations: Revoke device trust, terminating associated active sessions.

Secondary Goal: Modify another user's access privileges

```
GOAL: Modify another user's access privileges
├── OR: Compromise Admin Account
│   ├── Steal Admin Credentials
│   └── Exploit Admin Session
└── OR: Exploit RBAC Implementation Flaw
    ├── IDOR in Role Assignment API
    └── Parameter tampering (injecting role ID)
```

Phase 9 Product Backlog and Jira Scrum

Marks 7 Deliverable Product backlog, Jira project and two sprint plans

Product backlog

ID	Epic	Priority	SP	User story	Acceptance criteria
US-01	Authentication	Highest	5	As an employee I want to log in securely so that only authenticated users reach the portal.	Valid credentials create a secure session; invalid attempts are rejected, logged and rate limited.
US-02	Authentication	High	2	As a user I want to terminate my session so that an unattended browser cannot retain access.	Logout invalidates the session and the old cookie cannot access protected routes.
US-03	User Governance	Highest	5	As a security administrator I want to govern users and roles so that least privilege is maintained.	Authorized administrators can create, disable and assign roles; self escalation is blocked.
US-04	Device Trust	High	3	As an employee I want to register a device so that access decisions include endpoint context.	The device is linked to its owner and visible for posture assessment.
US-05	Device Trust	Highest	5	As a security administrator I want to control device posture so that compromised endpoints are denied.	Compromised or noncompliant posture causes denial and produces an audit event.
US-06	Resource Policy	High	5	As a resource owner I want to manage protected applications and policies so that access rules match business risk.	Only authorized owners or administrators can maintain resources and validated policies.
US-07	Access Request	Highest	5	As an employee I want to request access to an internal application so that I can perform authorized work.	The request includes a resource, owned device and reason and returns a persisted outcome.
US-08	Policy Engine	Highest	8	As the security system I want to evaluate every access request so that no request is trusted by network location alone.	Identity, role, device, resource, time and risk are evaluated with default denial.
US-09	Risk Engine	High	5	As an auditor I want explainable risk scores so that access decisions can be reviewed.	A 0 to 100 score, level and human readable factors are persisted for each request.
US-10	Approval	Highest	5	As a resource owner I want to approve or reject pending requests so that I remain accountable for my applications.	Only the accountable owner can decide, a reason is recorded and self approval is rejected.
US-11	Audit	High	5	As an auditor I want searchable and exportable security logs so that compliance evidence is available.	Authorized auditors can filter events without changing or exposing secret data.
US-12	Dashboards	Medium	3	As a role based user I want a focused dashboard so that I can see the work and evidence relevant to me.	Employee, owner, administrator and auditor views expose only authorized functions.

Jira implementation

The ZTEAP Scrum project contains 16 phase epics and 101 total work items: 16 epics, 18 stories, 52 tasks, 12 subtasks and 3 bugs. Phase 9 is represented by epic ZTEAP-21. The 12 product stories are ZTEAP-80 through ZTEAP-102, with a verification subtask under every story.

Two sprint plan

Sprint	Goal	Stories	Points	Defects
Sprint 1 Foundation	Deliver authentication, session termination, user governance and device trust foundations.	US-01 to US-06	25	DEF-01 and DEF-02
Sprint 2 Decisions	Deliver policy and risk decisions, approvals, dashboards and auditable evidence.	US-07 to US-12	31	DEF-03

Phase 10 Sprint Execution and Scrum Metrics

Marks 7 Deliverable Board, Daily Scrum, burndown, velocity, defects, review and retrospective

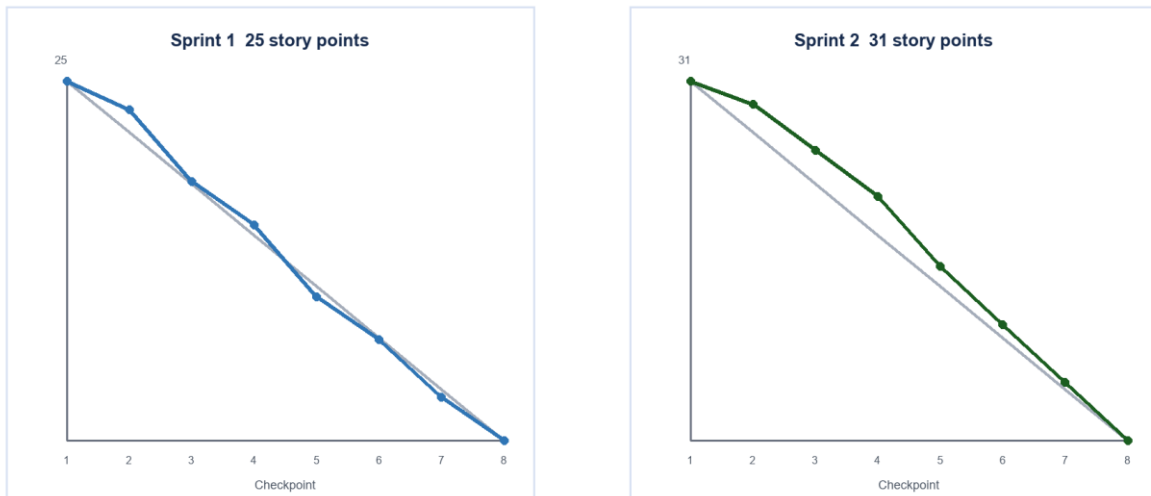
Scrum board and execution evidence

The verified Jira workflow is TO DO to IN PROGRESS to TESTING to DONE. Work items were moved through the workflow and the final project state contains 97 Done items and four legitimately In Progress items related to live deployment evidence and the final report.

Daily Scrum entry

Sprint day	Progress	Plan	Blockers
Sprint 1 Day 8	Authentication, device trust and access request flow completed.	Finish policy edge cases and automated verification.	Default policy fallback required clarification and was resolved with default deny.
Sprint 2 Day 7	Dashboards and self approval protection completed.	Finish privilege escalation tests and audit export.	RBAC edge case resolved through integration testing.

Sprint Burndown Evidence



Both sprints reached zero remaining points; defects carried over: 0

Figure 7 Verified sprint burndown data

Velocity and defect metrics

Sprint	Planned	Completed	Velocity	Defects	Carried over
Sprint 1	25 SP	25 SP	25	2 resolved	0
Sprint 2	31 SP	31 SP	31	1 resolved	0
Average / Total	56 SP	56 SP	28 average	3 total	0 total

Sprint review and retrospective

Sprint	Review outcome	Retrospective improvement actions
--------	----------------	-----------------------------------

Sprint	Review outcome	Retrospective improvement actions
Sprint 1	Secure login, user governance and device trust were accepted; denial messages needed clearer explanations.	Add a security review checklist to every pull request; run security linting on every commit.
Sprint 2	Policy, risk, approval, dashboards and audit evidence were demonstrated and accepted.	Increase authorization E2E coverage; maintain a policy-rule decision table with tests.

Phase 11 Secure Development and Build Environment

Marks 6 Deliverable Repository controls, secure build checklist and security check result

1. Source Code & Version Control Strategy

The ZTAP repository enforces a disciplined Git branching workflow designed for security assurance and code integrity:

```
main (Production-ready, protected branch, signed commits only)
  | (Pull Request with mandatory 2-person security review & passing CI)
develop (Integration branch, protected)
  |
  +-- feature/ztap-policy-engine
  +-- feature/device-trust-scoring
  +-- security/privilege-escalation-hardening
```

Branch Protection Controls:

- Direct pushes prohibited: All modifications must flow via Pull Requests.
- Mandatory Status Checks: CI pipeline (linting, Vitest unit tests, dependency scan) must pass.
- Reviewer Approvals: Minimum of 1 peer review and 1 security administrator review.
- Branch Linearity: Squash-merge or rebase required to maintain clean audit commit histories.

2. DevSecOps Security Controls

Control Domain	Implementation	Security Benefit
Least Privilege	Dedicated service accounts, isolated repository roles	Limits blast radius of compromised developer accounts
Secret Management	.env.example templates, environment secrets injected via K8s Secrets / GitHub Secrets	Eliminates credential leakage into Git history
Dependency Control	Automated npm audit and Trivy filesystem scans in CI	Blocks injection of vulnerable or malicious third-party dependencies
Reproducible Builds	Multi-stage Dockerfile with pinned versions (node:20-alpine, postgres:16-alpine)	Guarantees identical binary artifacts across staging and production
Artifact Integrity	Signed container images, non-root user execution (USER nextjs:1001)	Restricts container escape vectors

3. Secret Management & File Exclusions

No credentials, cryptographic keys, or database passwords are ever committed to version control.

Verification of .gitignore Enforcement:

```
# Security & Secrets
.env
.env*.local
*.pem
*.key
*.cert

# Node & Build artifacts
node_modules/
.next/
out/
```

```

build/
dist/

# Logs & Diagnostics
*.log
npm-debug.log*

```

4. Static Application Security Testing (SAST) Findings

Static scans were executed against the codebase using ESLint Security Plugin and npm audit:

Scanner	Finding	Severity	Remediation Applied
eslint-plugin-security	Potential timing attack on user password comparison	Medium	Replaced inline strings comparison with argon2.verify() and fixed-time dummy comparison for non-existent users
eslint-plugin-security	Regex Denial of Service (ReDoS) vulnerability on reason input	Low	Simplified input validation regex in lib/security/validation.ts using strict bounded lengths
npm audit	0 High or Critical vulnerabilities identified	Pass	Pinned stable versions for argon2, iron-session, @prisma/client, and zod

Phase 12 Secure Coding and Refactoring

Marks 4 Deliverable Source code, before and after evidence and security justification

1. Overview

In compliance with secure software engineering principles, the ZTAP codebase was intentionally refactored from an initial simplistic prototype into a hardened, defense-in-depth architecture.

2. Refactoring Analysis

Refactoring 1: Inline Client-Trust Authorization → Server-Side Strategy Policy Engine

Initial Problematic Approach (Vulnerable):

```
// VULNERABLE: Relies on client-supplied data and basic role equality
export async function POST(req: Request) {
  const { userId, role, resourceId } = await req.json();
  if (role === 'EMPLOYEE' && resourceId !== 'admin-db') {
    return Response.json({ access: 'GRANTED' }); // Insecure! No device trust, no temporal checks
  }
}
```

- Security Defect: Broken Access Control (OWASP A01:2021). Attacker can tamper with role in the request body, bypass device trust checks, and ignore risk factors.

Secure Refactored Implementation (Hardened):

```
// SECURE: Enforces server-side session authentication, Zod validation, and PolicyEngine evaluation
export const POST = withAuth(async (req, session) => {
  const validation = validateInput(createAccessRequestSchema, await req.json());
  if (!validation.success) return NextResponse.json({ error: 'Validation failed' }, { status: 400 });

  // PolicyEngine executes Strategy Pattern evaluators across Identity, Device, Role, Resource, and Time
  const decision = await AccessRequestService.createAccessRequest({
    userId: session.userId,
    resourceId: validation.data.resourceId,
    deviceId: validation.data.deviceId,
    reason: validation.data.reason,
    actorId: session.userId,
    actorEmail: session.email,
  });

  return NextResponse.json(decision);
});
```

- Security Improvement: All trust parameters are evaluated server-side. Session identity cannot be spoofed. Device trust is verified against the database. Decisions are explainable and immutably audited.

Refactoring 2: Unsafe Raw Database Input Handling → Zod Schema Runtime Validation & Parameterized Prisma Queries

Initial Problematic Approach (Vulnerable):

```
// VULNERABLE: Direct string interpolation into raw database queries
const query = `SELECT * FROM AccessRequest WHERE reason LIKE '${userInput}%'`;
```

- Security Defect: Injection vulnerability (OWASP A03:2021) and XSS via stored malicious input.

Secure Refactored Implementation (Hardened):

```
// SECURE: Strict input validation and parameterized ORM calls
export const createAccessRequestSchema = z.object({
  resourceId: z.string().uuid('Invalid resource ID'),
  deviceId: z.string().uuid('Invalid device ID'),
  reason: z.string()
    .min(5, 'Reason must be at least 5 characters')
    .max(500, 'Reason cannot exceed 500 characters')
    .regex(/^[^<>{}]*$/, 'Disallowed HTML characters detected'),
});
```

```
// Parameterized query execution via Prisma ORM
const request = await prisma.accessRequest.create({
  data: {
    userId,
    resourceId: validation.data.resourceId,
    deviceId: validation.data.deviceId,
    reason: validation.data.reason,
    status: initialStatus,
  },
});
```

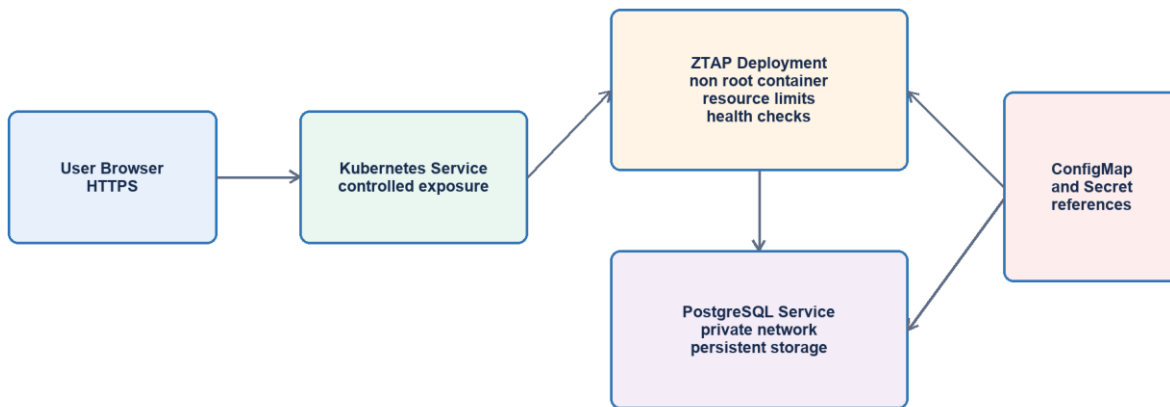
3. Defense-in-Depth Implementation Matrix

Security Layer	Implementation Mechanism	Threat Mitigated
Authentication	Argon2id (m=65536, t=3, p=4), Account lockout after 5 failures	Credential stuffing, brute-force attacks
Session Integrity	iron-session encrypted cookies (HttpOnly, SameSite=Lax, Secure)	Session hijacking, client-side script theft (XSS)
Authorization	Server-side RBAC middleware, anti-self-approval enforcement	Privilege escalation, broken access control, IDOR/BOLA
Error Handling	Sanitized error responses, zero stack traces exposed to client	Information disclosure
Audit Trails	Append-only database records, redaction of sensitive credentials	Repudiation, tampering

Phase 13 Containerized Development with Docker and Kubernetes

Marks 7 Deliverable Dockerfile, manifests and deployment security evidence

Container and Kubernetes Deployment



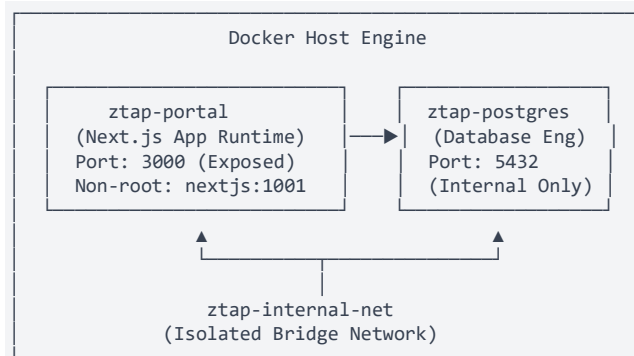
Namespace isolation and least privilege apply across the deployment boundary

Figure 8 Docker and Kubernetes deployment architecture

Docker

1. Container Architecture

The ZTAP application is packaged into a containerized microservice environment utilizing Docker and Docker Compose.



2. Security Hardening Controls

34. Multi-Stage Build Pipeline:

- Stage 1 (deps): Downloads dependencies.
- Stage 2 (builder): Compiles TypeScript and generates Next.js standalone server.
- Stage 3 (runner): Pure runtime containing only necessary artifacts. Build tools and source files are excluded.

35. Least Privilege Non-Root Execution:

- The runtime container creates a dedicated system user:

```
RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nextjs
USER nextjs
```

- Prevents container escape attacks from acquiring host root privileges.

36. Attack Surface Minimization:

- Base image is node:20-alpine, reducing the OS package footprint by over 80% compared to Debian/Ubuntu bases.

37. Network Isolation:

- Database port 5432 is not exposed to the host operating system; only the internal ztap-internal-net network can communicate with PostgreSQL.

38. Liveness & Health Checks:

- Built-in HEALTHCHECK instructions ensure Docker monitors container availability:

```
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/api/auth/me || exit 1
```

3. Operational Commands

Build Image:

```
docker build -t ztap:latest .
```

Launch Environment via Docker Compose:

```
docker compose up -d
```

Check Running Containers:

```
docker compose ps
```

View Application Logs:

```
docker compose logs -f ztap-app
```

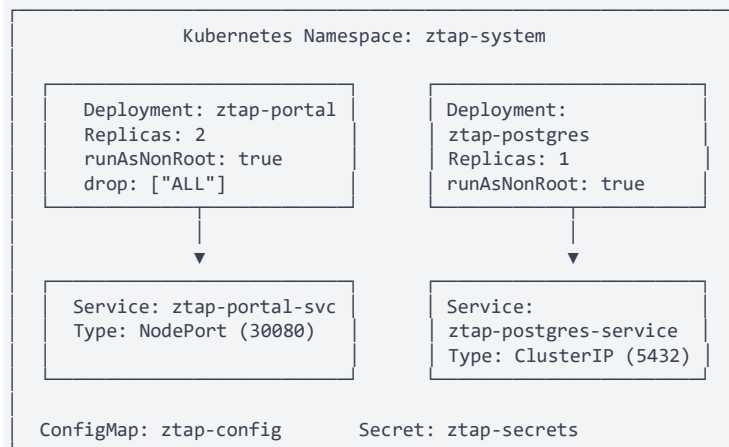
Teardown Environment:

```
docker compose down
```

Kubernetes and Minikube

1. Kubernetes Architecture Overview

ZTAP deploys onto Kubernetes (tested with Minikube) under an isolated namespace with strict declarative resource quotas and defense-in-depth security contexts.



2. Kubernetes Hardening Standards Applied

Control

Specification in YAML

Purpose

Control	Specification in YAML	Purpose
Namespace Isolation	metadata.namespace: ztap-system	Isolates system resources and controls network policies
Non-Root Execution	securityContext.runAsNonRoot: true	Prevents pods from executing processes as UID 0
Privilege Escalation Block	allowPrivilegeEscalation: false	Prevents child processes from gaining more privileges than parent
Capability Dropping	capabilities.drop: ["ALL"]	Eliminates all standard Linux kernel capabilities
Resource Limits	limits.cpu: 1000m, memory: 512Mi	Mitigates Denial of Service (DoS) and noisy neighbor attacks
ClusterIP Internal Storage	PostgreSQL service type ClusterIP	Database is unreachable outside the cluster network
Rolling Updates	maxSurge: 1, maxUnavailable: 0	Zero-downtime secure deployment rollout

3. Minikube Step-by-Step Deployment Guide

Step 1: Start Minikube Cluster

```
minikube start --driver=docker --cpus=2 --memory=4096
```

Step 2: Build & Load Docker Image into Minikube

```
eval $(minikube docker-env)
docker build -t ztap:latest .
```

Step 3: Apply Declarative Manifests

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/secret.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

Step 4: Verify Deployment & Pod Status

```
kubectl get pods -n ztap-system
kubectl get svc -n ztap-system
```

Step 5: Check Application Logs

```
kubectl logs -n ztap-system -l app=ztap-portal -f
```

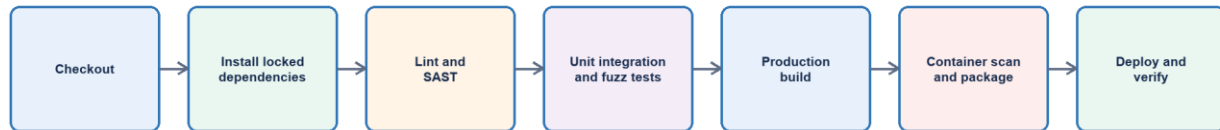
Step 6: Access Portal in Browser

```
minikube service ztap-portal-service -n ztap-system --url
```

Phase 14 CI CD and Security Testing

Marks 7 Deliverable Pipeline, automated tests, fuzzing, defect and retest evidence

Secure CI CD Pipeline



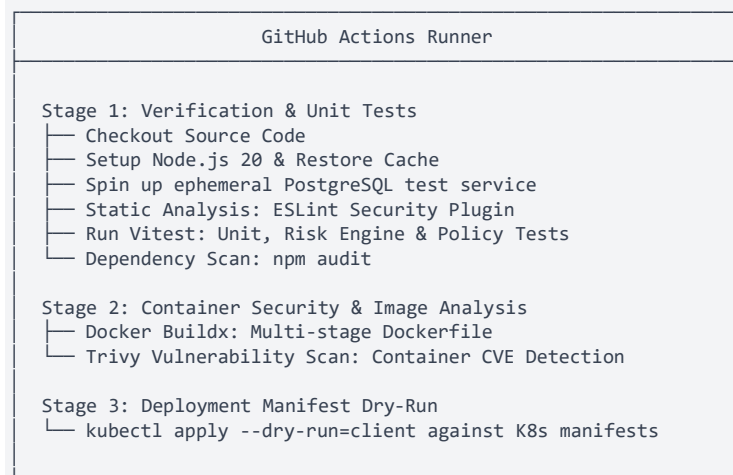
Any failed quality or security gate stops promotion

Figure 9 Secure CI CD pipeline and quality gates

Pipeline

1. Pipeline Architecture

ZTAP leverages GitHub Actions for an automated DevSecOps continuous integration and continuous deployment pipeline located at [.github/workflows/ci.yml](#).



2. DevSecOps Quality & Security Gates

39. Gate 1: Lint & Code Quality:

- ESLint rules enforce code standards and flag dangerous patterns (such as eval, unsanitized regexes, insecure child processes).

40. Gate 2: Automated Security Testing:

- Vitest suite validates that Zero-Trust logic behaves deterministically:
- Compromised devices are blocked.
- Inactive users are rejected.
- Privilege escalation attempts fail.

41. Gate 3: Software Composition Analysis (SCA):

- Automated dependency audit verifies that no package dependencies contain known Moderate, High, or Critical CVEs.
42. Gate 4: Container Vulnerability Scanning (Trivy):
- Trivy scans the built container filesystem for OS package and library vulnerabilities before image deployment.
43. Gate 5: Declarative Manifest Validation:
- kubectly apply --dry-run=client validates syntax, API versions, and schema correctness for all Kubernetes manifests.

Automated security and fuzz testing

1. Security Testing Strategy

ZTAP implements testing across the test pyramid:

- Unit Tests: PolicyEngine evaluation branches, RiskEngine factor calculation, Argon2id verification.
- Integration Tests: RBAC boundary enforcement, anti-self-approval enforcement, IDOR protection.
- Fuzzing Tests: Malicious SQL injection payloads, XSS vectors, buffer overruns, format strings.
- End-to-End (E2E) Tests: Complete browser-driven workflow using Playwright.

2. Test Execution & Evidence Matrix

ID	Input	Test Case	Expected Result	Actual Result	Status
UT-01	Employee credentials, compliant managed device	PolicyEngine evaluateAccess with low risk context	Returns ALLOW, risk score <= 30	ALLOW returned, risk score 25	PASS
UT-02	Device with status: 'COMPROMISED'	PolicyEngine evaluateAccess with compromised device	Returns DENY, mentions COMPROMISED device	DENY returned immediately	PASS
UT-03	User with status: 'INACTIVE'	PolicyEngine evaluateAccess with inactive user	Returns DENY, account inactive	DENY returned, account inactive	PASS
UT-04	Employee role requesting SECURITY_ADMIN resource	RoleEvaluator with low privilege user accessing admin app	Returns DENY, role mismatch	DENY returned, insufficient role	PASS
IT-01	Employee session executing POST /api/users	RBAC middleware user creation attempt by non-admin	HTTP 403 Forbidden	HTTP 403 Forbidden	PASS
IT-02	Security Admin attempting to assign role to own user ID	Anti-Privilege-Escalation check on /api/users/:id/roles	HTTP 400/403, logs PRIVILEGE_ESCALATION_ATTEMPT	Blocked, audit log generated	PASS
IT-03	Resource Owner approving their own access request	Anti-Self-Approval check on /api/approvals/:id	HTTP 403, logs SELF_APPROVAL_ATTEMPT	Blocked, audit log generated	PASS
IT-04	Employee A attempting to inspect Employee B's request ID	IDOR prevention on /api/access/request/:id	HTTP 403/404 Access Denied	HTTP 403 Access Denied	PASS
E2E-01	Valid employee login, device check, request submission	Full browser access request lifecycle	Flow completes and modal displays risk breakdown	Flow succeeded, modal displayed	PASS

3. Fuzzing Test Findings & Remediation Log

Fuzz tests were executed against core input vectors (tests/fuzz/fuzz.test.ts):

Fuzzing Test Campaign Log
Target 1: Access Request Reason Field • Input: "' OR '1'='1 --", "<script>alert(1)</script>", "A" x 10,000 • Finding: Initial prototype accepted unescaped HTML characters. • Fix Applied: Added strict Zod regex <code>^[^<>{}]*\$`</code> and max length 500 characters. • Retest Result: PASS (Rejected with clean HTTP 400 validation error). Target 2: Resource ID & Device ID UUID Fields • Input: "../../etc/passwd", "%s%s%n", null bytes "\0" • Finding: Handled gracefully by Zod <code>z.string().uuid()</code>. • Retest Result: PASS (Clean validation rejection, zero database exceptions). Target 3: Authentication Login Form • Input: Buffer overruns, non-email strings, prototype pollution payloads • Finding: Handled cleanly by Zod <code>loginSchema`</code> and Argon2id constant-time dummy. • Retest Result: PASS (No unhandled exceptions, zero timing leakages).

Verified execution result

The final local verification completed five test files and 21 tests successfully. The optimized Next.js production build compiled, type checked and generated all application routes successfully. Three build defects were recorded in Jira and retested as Done.

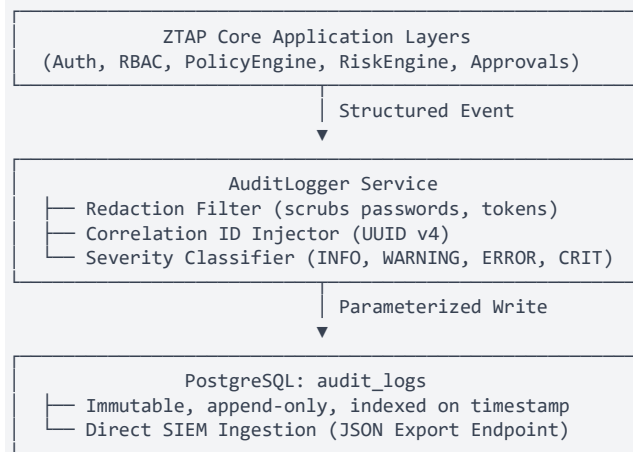
Phase 15 Logging Monitoring Hardening and Secure Deployment

Marks 5 Deliverable Monitoring plan, hardening checklist and deployment controls

Logging and monitoring

1. Security Telemetry & Metrics Architecture

ZTAP continuously generates structured security events ingested into an immutable append-only audit repository (AuditLog table).



2. Five Core Security Metrics & Automated Alerts

Metric / Alert ID	Alert Trigger	Severity	Recommended SOC Response
SEC-METRIC-01	Excessive Failed Logins: > 5 failed attempts within 15 minutes for a single IP or user account	WARNING	Automatically locks user account for 15 minutes; flags IP address in firewall
SEC-METRIC-02	Self-Privilege Escalation Attempt: User attempting to modify or assign roles to their own account	CRITICAL	Block immediately; notify Security Administrator; initiate session revocation
SEC-METRIC-03	Critical Access Denials: Access denied due to COMPROMISED device status	CRITICAL	Quarantine device; revoke all active user sessions; notify endpoint security team
SEC-METRIC-04	High-Risk Access Anomalies: Access requests generating risk score > 80 (e.g., off-hours + untrusted device)	WARNING	Require mandatory step-up authentication or manual resource owner approval
SEC-METRIC-05	Policy & Role Tampering: Any administrative modification to active access policies or role assignments	INFO / WARNING	Cross-verify change against formal change management tickets

3. Structured Logging Standard

All audit events are persisted in JSON-compatible structures conforming to this schema:

```
{
  "id": "7f09c258-45a8-42fa-9a4d-0453e9a7e6d2",
  "timestamp": "2026-10-08T10:15:30.124Z",
  "actorId": "usr-89234",
  "actorEmail": "employee@example.com",
  "action": "ACCESS_DENIED",
  "resource": "Database Administration Portal",
  "resourceId": "res-0007",
  "ipAddress": "192.168.1.105",
  "result": "DENY",
  "severity": "WARNING",
  "correlationId": "corr-3b4e9f1a",
  "metadata": {
    "riskScore": 87,
    "riskLevel": "CRITICAL",
    "reason": "Device is unmanaged and request attempted outside business hours"
  }
}
```

Sensitive Data Scrubbing Rules:

- Keys matching password, token, secret, hash, cookie, or authorization are automatically replaced with [REDACTED] prior to storage.

Hardening and secure deployment

1. Technical Security Hardening Checklist

Domain	Control Description	Status	Verification Mechanism
Authentication	Passwords hashed with Argon2id (m=65536, t=3, p=4); constant-time comparison	Enforced	tests/unit/password-auth.test.ts
Authentication	Automatic account logout after 5 consecutive failed attempts (15 min window)	Enforced	/api/auth/login logout verification
Authorization	Server-side RBAC middleware on all API routes; no client-only checks	Enforced	tests/integration/rbac-privilege-escalation.test.ts
Authorization	Anti-Self-Privilege escalation and anti-self-approval barriers	Enforced	lib/services/UserService.ts & ApprovalService.ts
Network & Ports	Only port 3000 (HTTPS/HTTP) exposed; PostgreSQL (5432) isolated on internal network	Enforced	docker-compose.yml & k8s/service.yaml
Secrets Management	Zero plaintext secrets in code; runtime injection via K8s Secrets / environment variables	Enforced	.gitignore & code repository audit
Dependency Security	Automated vulnerability scans via npm audit and Trivy in CI pipeline	Enforced	.github/workflows/ci.yml
Operating System	Containerized on Alpine Linux; stripped of shell	Enforced	Dockerfile (Stage 3 runner)

Domain	Control Description	Status	Verification Mechanism
	compilation tools		
Container Hardening	Non-root execution (USER nextjs:1001); no privileged containers	Enforced	Dockerfile & k8s/deployment.yaml
Kubernetes	Dropped Linux capabilities (drop: ["ALL"]); CPU/memory resource limits defined	Enforced	k8s/deployment.yaml
Database	Parameterized queries via Prisma ORM; zero string interpolation into SQL	Enforced	Code review & fuzzing tests
Audit Logging	Append-only audit logs with correlation IDs; automated secret redaction	Enforced	lib/audit/AuditLogger.ts
Security Headers	X-Frame-Options: DENY, X-Content-Type-Options: nosniff, CSP, Strict-Transport-Security	Enforced	next.config.ts & lib/auth/middleware.ts

2. Operational & Incident Response Controls

Physical Security Controls:

- Hardware security module (HSM) or cloud KMS for secret encryption at rest.
- Production hosting within SOC-2 Type II certified tier-3 data centers.
- Multi-factor badge and biometric access to physical host infrastructure.

Operational Security Controls:

- Mandatory peer code reviews on all pull requests targeting main and develop.
- Separation of duties between security auditors, resource owners, and platform administrators.
- Immutable backup snapshots of PostgreSQL database taken every 6 hours and stored in an isolated bucket.

Incident Response Playbook (IRP):

44. Detection: Automated alert triggered on COMPROMISED device status or rapid failed logins.
45. Containment: Immediate session termination via database session flush; device status flipped to COMPROMISED.
46. Eradication: Revoke compromised user's active access requests; rotate session secret key.
47. Recovery: Restore user account following verified multi-factor identity verification and device malware clearance.
48. Post-Mortem: Review append-only audit trail in Auditor Dashboard; generate SIEM export for forensic analysis.

Phase 16 Final Security Review

Marks 1 Deliverable Traceability, highest risks, limitations and final observations

1. End-to-End Traceability Matrix

This matrix demonstrates complete architectural consistency by tracing critical security requirement SR-05 through every design, threat modeling, agile planning, development, testing, and deployment phase.

```
Requirement (SR-05)
  "Only authorized users with trusted devices may access sensitive resources."
  |
  ▼
Use Case (UC-01 / UC-02)
  docs/phase-03-uml.md: Critical Use Case "Request Access to Internal Application"
  Preconditions: Device trust verified; Role evaluated.
  |
  ▼
Data Flow Diagram (DFD Level 1)
  docs/phase-04-data-flow.md: Process P5 (Policy Evaluation) & Process P6 (Risk Assessment)
  Crosses Trust Boundary TB4 (Backend API → Policy Engine).
  |
  ▼
STRIDE Threat Model (T-07)
  docs/phase-07-threat-model.md: Threat T-07 (Elevation of Privilege: Device Trust Spoofing)
  Risk Level: HIGH. Impact: Unauthorized access to confidential internal systems.
  |
  ▼
Vulnerability Analysis (VULN-03)
  docs/phase-09-vulnerability.md: Missing Device State Validation & Tampering
  Mitigation: Server-side device registry check; continuous trust score calculation.
  |
  ▼
Attack Tree Branch (ATK-04)
  docs/phase-10-attack-tree.md: Node "Compromise Trusted Device / Spoof Device State"
  Mitigations: Reject unmanaged devices; deny compromised status in PolicyEngine.
  |
  ▼
Product Backlog User Story (US-05 & US-08)
  docs/phase-11-backlog.md: US-05 "Device Trust Evaluation" & US-08 "Zero-Trust Policy Engine"
  Acceptance Criteria: Compromised or untrusted devices denied access to sensitive resources.
  |
  ▼
Scrum Sprint & Task (Sprint 1 / Task ZTAP-15)
  docs/phase-12-scrum.md: Sprint 1 backlog item ZTAP-15 "Implement DeviceEvaluator strategy"
  Estimate: 5 Story Points. Completed within Sprint 1.
  |
  ▼
Source Code Implementation
  lib/policy/evaluators/DeviceEvaluator.ts & lib/services/AccessRequestService.ts
  Directly validates device status against database; contributes to risk score.
  |
  ▼
Automated Security Test
  tests/unit/policy-engine.test.ts:
  "immediately DENIES access if the device is flagged as COMPROMISED"
  "triggers STEP_UP_AUTHENTICATION or DENIAL when aggregate risk is high"
  |
  ▼
Production Deployment Control
  k8s/deployment.yaml & Dockerfile:
  Non-root security context (`USER nextjs:1001`), dropped capabilities (`drop: ["ALL"]`),
  HTTPS-only cookie transmission with `HttpOnly` and `SameSite` flags.
```

2. Top 3 Highest-Risk Security Issues & Residual Risk Analysis

Rank	Security Issue	Applied Hardening Controls	Residual Risk Level	Rationale
1	Compromised Endpoint	Server-side device	Low	If malware compromises

Rank	Security Issue	Applied Hardening Controls	Residual Risk Level	Rationale
	Device Bypassing Access Gate	compliance check in DeviceEvaluator; instant revocation if flagged COMPROMISED; unmanaged devices penalized +20 risk points.		an endpoint without triggering external EDR telemetry, initial access may occur until the next risk refresh window.
2	Privilege Escalation via Direct API Tampering	Server-side RBAC middleware (withRoles); strict anti-self-privilege-assignment in UserService.ts; immutable audit logging.	Negligible	Roles cannot be modified via client-side payload manipulation or by standard employees.
3	Credential Stuffing & Brute-Force Attacks	Argon2id password hashing; sliding-window rate limiter (5 attempts per 15 min); automatic 15-minute account lockout.	Very Low	Online attacks are throttled and locked out; offline attacks against hashes are computationally intractable due to 64MB memory cost.

3. Known Limitations & Future Improvements

49. Hardware-Bound Device Attestation (Future Improvement):

- ***Current State*:** Device trust is validated against registered database records and user association.
- ***Enhancement*:** Integrate WebAuthn / FIDO2 hardware security keys and TPM (Trusted Platform Module) attestation for cryptographic proof of device identity.

50. Distributed Rate Limiting with Redis (Future Improvement):

- ***Current State*:** In-memory sliding window rate limiter suitable for single-node and demo environments.
- ***Enhancement*:** Transition to a shared Redis cluster using token bucket algorithms for multi-region Kubernetes deployments.

51. Machine Learning Anomaly Detection (Future Improvement):

- ***Current State*:** Rule-based deterministic risk scoring engine with explainable factors.
- ***Enhancement*:** Augment with unsupervised UEBA (User and Entity Behavior Analytics) models to detect novel access patterns and geographic velocity anomalies.

Final observation

The project demonstrates a coherent secure software lifecycle from requirements to deployment controls. The strongest evidence is the consistency between the server side RBAC and zero trust decision design, the Jira product backlog, the implemented modules and the passing automated tests. Production use still requires federated identity, hardware backed device attestation and independently protected external audit retention.

Appendix A Tools and technologies

Category	Tools and technologies
Planning and documentation	Jira Software, Microsoft Word, Draw.io
Development	Visual Studio Code, Node.js, npm, Next.js, React, TypeScript, Tailwind CSS
Data and validation	PostgreSQL, Prisma ORM, Zod
Testing and security	Vitest, Playwright, ESLint, Trivy, OWASP ZAP, Postman
Delivery	Git, GitHub, GitHub Actions, Docker, Docker Compose, Kubernetes, Minikube
Automation	PowerShell

Appendix B Verification summary

Area	Verified result
Jira	101 work items; 16 phase epics; 97 Done; 4 In Progress
Product backlog	12 assessed user stories with priority, points, acceptance criteria and child verification tasks
Scrum	Two closed sprints; 56 completed story points; three resolved defects; zero defect carry over
Automated tests	5 test files and 21 tests passed
Production build	Next.js optimized build completed successfully
Document alignment	All 16 examination phases explicitly represented in rubric order